

Skill-3

ORDERS TABLE

STEP 1: CREATE TABLE

```
CREATE TABLE orders (  
    ord_no INT PRIMARY KEY,  
    purch_amt DECIMAL(10,2),  
    ord_date DATE,  
    customer_id INT,  
    salesman_id INT  
);
```

```
mysql> CREATE TABLE orders (  
->     ord_no INT PRIMARY KEY,  
->     purch_amt DECIMAL(10,2),  
->     ord_date DATE,  
->     customer_id INT,  
->     salesman_id INT  
-> );  
Query OK, 0 rows affected (0.48 sec)
```

STEP 2: INSERT DATA

```
INSERT INTO orders VALUES  
(70001,150.50,'2012-10-05',3005,5002),  
(70009,270.65,'2012-09-10',3001,5005),  
(70002,65.26,'2012-10-05',3002,5001),  
(70004,110.50,'2012-08-17',3009,5003),  
(70007,948.50,'2012-09-10',3005,5002),  
(70005,2400.60,'2012-07-27',3007,5001),  
(70008,5760.00,'2012-09-10',3002,5001),  
(70010,1983.43,'2012-10-10',3004,5006),  
(70003,2480.40,'2012-10-10',3009,5003),  
(70012,250.45,'2012-06-27',3008,5002),  
(70011,75.29,'2012-08-17',3003,5007),  
(70013,3045.60,'2012-04-25',3002,5001);
```

```
mysql> INSERT INTO orders VALUES
-> (70001,150.50,'2012-10-05',3005,5002),
-> (70009,270.65,'2012-09-10',3001,5005),
-> (70002,65.26,'2012-10-05',3002,5001),
-> (70004,110.50,'2012-08-17',3009,5003),
-> (70007,948.50,'2012-09-10',3005,5002),
-> (70005,2400.60,'2012-07-27',3007,5001),
-> (70008,5760.00,'2012-09-10',3002,5001),
-> (70010,1983.43,'2012-10-10',3004,5006),
-> (70003,2480.40,'2012-10-10',3009,5003),
-> (70012,250.45,'2012-06-27',3008,5002),
-> (70011,75.29,'2012-08-17',3003,5007),
-> (70013,3045.60,'2012-04-25',3002,5001);
Query OK, 12 rows affected (0.07 sec)
Records: 12  Duplicates: 0  Warnings: 0
```

QUESTION 1 - WHERE CLAUSE

SCENARIO:

Finance wants all orders whose purchase amount is greater than \$2,000.

QUERY:

```
SELECT *
FROM orders
WHERE purch_amt > 2000;
```

```
mysql> SELECT *
-> FROM orders
-> WHERE purch_amt > 2000;
+-----+-----+-----+-----+-----+
| ord_no | purch_amt | ord_date | customer_id | salesman_id |
+-----+-----+-----+-----+-----+
| 70003 | 2480.40 | 2012-10-10 | 3009 | 5003 |
| 70005 | 2400.60 | 2012-07-27 | 3007 | 5001 |
| 70008 | 5760.00 | 2012-09-10 | 3002 | 5001 |
| 70013 | 3045.60 | 2012-04-25 | 3002 | 5001 |
+-----+-----+-----+-----+-----+
4 rows in set (0.04 sec)
```

QUESTION 2 - WHERE CLAUSE

SCENARIO:

Management wants all orders placed on 2012-09-10.

QUERY:

```
SELECT *
```

```
FROM orders
WHERE ord_date='2012-09-10';
```

```
mysql> SELECT *
-> FROM orders
-> WHERE ord_date='2012-09-10';
```

ord_no	purch_amt	ord_date	customer_id	salesman_id
70007	948.50	2012-09-10	3005	5002
70008	5760.00	2012-09-10	3002	5001
70009	270.65	2012-09-10	3001	5005

```
3 rows in set (0.01 sec)
```

QUESTION 3 - WHERE CLAUSE

SCENARIO:

Find all orders handled by salesman 5001.

QUERY:

```
SELECT *
FROM orders
WHERE salesman_id=5001;
```

```
mysql> SELECT *
-> FROM orders
-> WHERE salesman_id=5001;
```

ord_no	purch_amt	ord_date	customer_id	salesman_id
70002	65.26	2012-10-05	3002	5001
70005	2400.60	2012-07-27	3007	5001
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001

```
4 rows in set (0.01 sec)
```

QUESTION 4 - ORDER BY

SCENARIO:

CEO wants to see orders ranked from highest amount to lowest amount.

QUERY:

```
SELECT *
FROM orders
ORDER BY purch_amt DESC;
```

```
mysql> SELECT *
-> FROM orders
-> ORDER BY purch_amt DESC;
```

ord_no	purch_amt	ord_date	customer_id	salesman_id
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001
70003	2480.40	2012-10-10	3009	5003
70005	2400.60	2012-07-27	3007	5001
70010	1983.43	2012-10-10	3004	5006
70007	948.50	2012-09-10	3005	5002
70009	270.65	2012-09-10	3001	5005
70012	250.45	2012-06-27	3008	5002
70001	150.50	2012-10-05	3005	5002
70004	110.50	2012-08-17	3009	5003
70011	75.29	2012-08-17	3003	5007
70002	65.26	2012-10-05	3002	5001

```
12 rows in set (0.01 sec)
```

QUESTION 5 - ORDER BY

SCENARIO:

Operations team wants oldest orders first.

QUERY:

```
SELECT *
FROM orders
ORDER BY ord_date;
```

```
mysql> SELECT *
-> FROM orders
-> ORDER BY ord_date;
```

ord_no	purch_amt	ord_date	customer_id	salesman_id
70013	3045.60	2012-04-25	3002	5001
70012	250.45	2012-06-27	3008	5002
70005	2400.60	2012-07-27	3007	5001
70004	110.50	2012-08-17	3009	5003
70011	75.29	2012-08-17	3003	5007
70007	948.50	2012-09-10	3005	5002
70008	5760.00	2012-09-10	3002	5001
70009	270.65	2012-09-10	3001	5005
70001	150.50	2012-10-05	3005	5002
70002	65.26	2012-10-05	3002	5001
70003	2480.40	2012-10-10	3009	5003
70010	1983.43	2012-10-10	3004	5006

12 rows in set (0.01 sec)

QUESTION 6 - SUM()

SCENARIO:

Finance department wants total revenue generated.

QUERY:

```
SELECT SUM(purch_amt) AS total_revenue
FROM orders;
```

```
mysql> SELECT SUM(purch_amt) AS total_revenue
-> FROM orders;
+-----+
| total_revenue |
+-----+
|      17541.18 |
+-----+
1 row in set (0.03 sec)
```

QUESTION 7 - AVG()

SCENARIO:

Management wants average order value.

QUERY:

```
SELECT AVG(purch_amt) AS average_order
FROM orders;
```

```
mysql> SELECT AVG(purch_amt) AS average_order
-> FROM orders;
+-----+
| average_order |
+-----+
| 1461.765000 |
+-----+
1 row in set (0.00 sec)
```

QUESTION 8 - MAX()

SCENARIO:

Find the highest order amount.

QUERY:

```
SELECT MAX(purch_amt) AS highest_order
FROM orders;
```

```
mysql> SELECT MAX(purch_amt) AS highest_order
-> FROM orders;
+-----+
| highest_order |
+-----+
| 5760.00 |
+-----+
1 row in set (0.01 sec)
```

QUESTION 9 - MIN()

SCENARIO:

Find the lowest order amount.

QUERY:

```
SELECT MIN(purch_amt) AS lowest_order
FROM orders;
```

```
mysql> SELECT MIN(purch_amt) AS lowest_order
-> FROM orders;
+-----+
| lowest_order |
+-----+
| 65.26 |
+-----+
1 row in set (0.00 sec)
```

QUESTION 10 - COUNT()

SCENARIO:

Find total number of orders.

QUERY:

```
SELECT COUNT(*) AS total_orders
FROM orders;
```

```
mysql> SELECT COUNT(*) AS total_orders
-> FROM orders;
+-----+
| total_orders |
+-----+
|          12 |
+-----+
1 row in set (0.06 sec)
```

QUESTION 11 - GROUP BY

SCENARIO:

Sales director wants total sales generated by each salesman.

QUERY:

```
SELECT salesman_id,
       SUM(purch_amt) AS total_sales
FROM orders
GROUP BY salesman_id;
```

```
mysql> SELECT salesman_id,
->          SUM(purch_amt) AS total_sales
-> FROM orders
-> GROUP BY salesman_id;
+-----+-----+
| salesman_id | total_sales |
+-----+-----+
|          5002 |      1349.45 |
|          5001 |     11271.46 |
|          5003 |      2590.90 |
|          5005 |       270.65 |
|          5006 |      1983.43 |
|          5007 |        75.29 |
+-----+-----+
6 rows in set (0.01 sec)
```

QUESTION 12 - GROUP BY

SCENARIO:

Marketing wants total purchases made by each customer.

QUERY:

```
SELECT customer_id,  
       SUM(purch_amt) AS total_purchase  
FROM orders  
GROUP BY customer_id;
```

```
mysql> SELECT customer_id,  
->      SUM(purch_amt) AS total_purchase  
-> FROM orders  
-> GROUP BY customer_id;  
+-----+-----+  
| customer_id | total_purchase |  
+-----+-----+  
|          3005 |          1099.00 |  
|          3002 |          8870.86 |  
|          3009 |          2590.90 |  
|          3007 |          2400.60 |  
|          3001 |           270.65 |  
|          3004 |          1983.43 |  
|          3003 |            75.29 |  
|          3008 |           250.45 |  
+-----+-----+  
8 rows in set (0.01 sec)
```

QUESTION 13 - GROUP BY + MAX()

SCENARIO:

Find the highest order placed by each customer.

QUERY:

```
SELECT customer_id,  
       MAX(purch_amt) AS highest_purchase  
FROM orders  
GROUP BY customer_id;
```



```
mysql> SELECT customer_id,
->         MAX(purch_amt) AS highest_purchase
-> FROM orders
-> GROUP BY customer_id;
```

customer_id	highest_purchase
3005	948.50
3002	5760.00
3009	2480.40
3007	2400.60
3001	270.65
3004	1983.43
3003	75.29
3008	250.45

```
8 rows in set (0.00 sec)
```

QUESTION 14 - GROUP BY + HAVING

SCENARIO:

Company rewards salesmen whose total sales exceed \$3,000.

QUERY:

```
SELECT salesman_id,
        SUM(purch_amt) AS total_sales
FROM orders
GROUP BY salesman_id
HAVING SUM(purch_amt) > 3000;
```

```
mysql> SELECT salesman_id,
->         SUM(purch_amt) AS total_sales
-> FROM orders
-> GROUP BY salesman_id
-> HAVING SUM(purch_amt) > 3000;
```

salesman_id	total_sales
5001	11271.46

```
1 row in set (0.01 sec)
```

QUESTION 15 - GROUP BY + HAVING

SCENARIO:

Find customers whose total purchases exceed \$2,500.

QUERY:

```

SELECT customer_id,
       SUM(purch_amt) AS total_purchase
FROM orders
GROUP BY customer_id
HAVING SUM(purch_amt) > 2500;

```

```

mysql> SELECT customer_id,
->       SUM(purch_amt) AS total_purchase
-> FROM orders
-> GROUP BY customer_id
-> HAVING SUM(purch_amt) > 2500;
+-----+-----+
| customer_id | total_purchase |
+-----+-----+
|          3002 |          8870.86 |
|          3009 |          2590.90 |
+-----+-----+
2 rows in set (0.00 sec)

```

QUESTION 16 - GROUP BY + HAVING

SCENARIO:

Find customers who placed more than one order.

QUERY:

```

SELECT customer_id,
       COUNT(*) AS total_orders
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 1;

```

```

mysql> SELECT customer_id,
->       COUNT(*) AS total_orders
-> FROM orders
-> GROUP BY customer_id
-> HAVING COUNT(*) > 1;
+-----+-----+
| customer_id | total_orders |
+-----+-----+
|          3005 |             2 |
|          3002 |             3 |
|          3009 |             2 |
+-----+-----+
3 rows in set (0.00 sec)

```

QUESTION 17 - GROUP BY + HAVING + ORDER BY

SCENARIO:

Management wants top customers ranked by spending.

QUERY:

```
SELECT customer_id,  
       SUM(purch_amt) AS total_purchase  
FROM orders  
GROUP BY customer_id  
HAVING SUM(purch_amt) > 1000  
ORDER BY total_purchase DESC;
```

```
mysql> SELECT customer_id,  
->       SUM(purch_amt) AS total_purchase  
-> FROM orders  
-> GROUP BY customer_id  
-> HAVING SUM(purch_amt) > 1000  
-> ORDER BY total_purchase DESC;  
+-----+-----+  
| customer_id | total_purchase |  
+-----+-----+  
|          3002 |          8870.86 |  
|          3009 |          2590.90 |  
|          3007 |          2400.60 |  
|          3004 |          1983.43 |  
|          3005 |          1099.00 |  
+-----+-----+  
5 rows in set (0.03 sec)
```

QUESTION 18 - BETWEEN + HAVING

SCENARIO:

Find customers whose maximum purchase is between \$2,000 and \$6,000.

QUERY:

```
SELECT customer_id,  
       MAX(purch_amt) AS max_purchase  
FROM orders  
GROUP BY customer_id  
HAVING MAX(purch_amt) BETWEEN 2000 AND 6000;
```

```
mysql> SELECT customer_id,
->         MAX(purch_amt) AS max_purchase
-> FROM orders
-> GROUP BY customer_id
-> HAVING MAX(purch_amt) BETWEEN 2000 AND 6000;
```

customer_id	max_purchase
3002	5760.00
3009	2480.40
3007	2400.60

3 rows in set (0.01 sec)

QUESTION 19 - COUNT + HAVING

SCENARIO:

Find salesmen who handled at least two orders.

QUERY:

```
SELECT salesman_id,
        COUNT(*) AS total_orders
FROM orders
GROUP BY salesman_id
HAVING COUNT(*) >= 2;
```

```
mysql> SELECT salesman_id,
->         COUNT(*) AS total_orders
-> FROM orders
-> GROUP BY salesman_id
-> HAVING COUNT(*) >= 2;
```

salesman_id	total_orders
5002	3
5001	4
5003	2

3 rows in set (0.01 sec)

QUESTION 20 - MAX + GROUP BY + HAVING

SCENARIO:

Management wants daily highest purchases above \$2,000.

QUERY:

```
SELECT ord_date,
```

```
        MAX(purch_amt) AS highest_purchase
FROM orders
GROUP BY ord_date
HAVING MAX(purch_amt) > 2000;
```

```
mysql> SELECT ord_date,
->        MAX(purch_amt) AS highest_purchase
-> FROM orders
-> GROUP BY ord_date
-> HAVING MAX(purch_amt) > 2000;
```

ord_date	highest_purchase
2012-10-10	2480.40
2012-07-27	2400.60
2012-09-10	5760.00
2012-04-25	3045.60

```
4 rows in set (0.00 sec)
```

